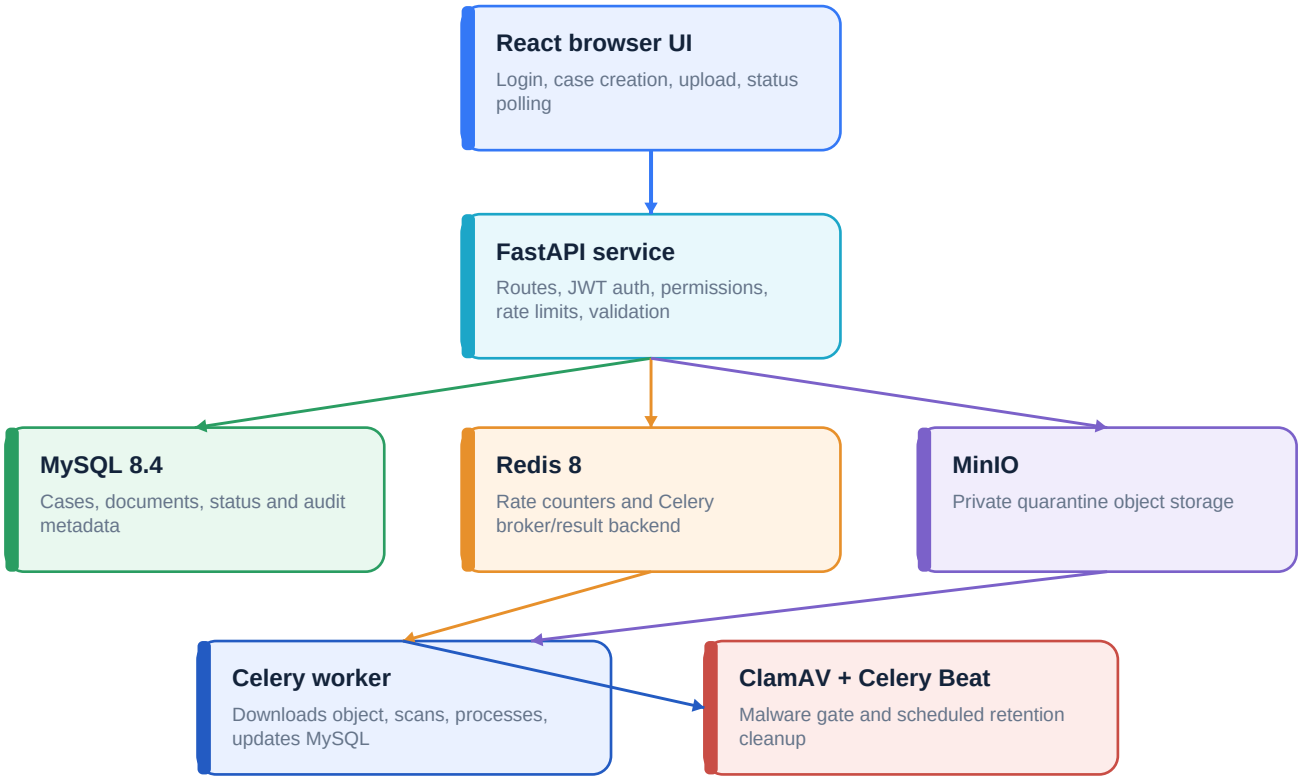
GRD Document Verification

Technical Architecture and Processing Guide

How an API request reaches the route, how authentication and rate limiting work, how Redis and Celery coordinate background jobs, how containers start, and how quarantine storage protects uploaded files.



Scope: local development and the Docker Compose deployment in grd-document-verf. Version: 1.0 | 08 August 2026

1. Technology stack

The application separates browser interaction, synchronous API controls, durable data, object storage, and asynchronous processing. This keeps uploads responsive while expensive security and document checks run outside the request thread.

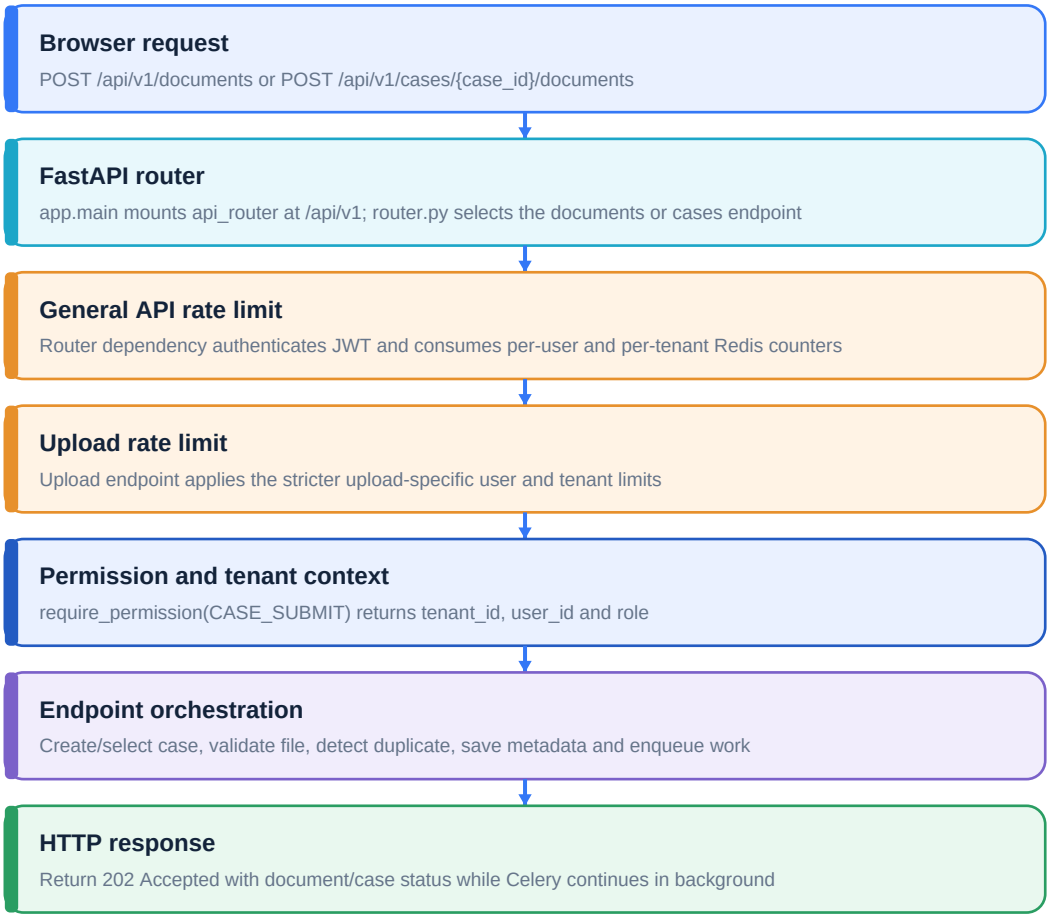
Layer	Technology	Responsibility
Frontend	React + TypeScript + Vite	Dashboard, login, document submission, case display and 3-second status polling.
API	FastAPI + Uvicorn	HTTP routing, validation, dependencies, authentication, tenant authorization and response models.
Data access	SQLAlchemy + PyMySQL	ORM models and transactions for MySQL-backed cases and document metadata.
Database	MySQL 8.4	Durable tenant-scoped cases, document hashes, statuses, storage keys and timestamps.
Queue and limits	Redis 8	Celery broker/result backend plus fixed-window request counters.
Background work	Celery	Runs malware scanning and the document-processing pipeline outside the API request.
Scheduler	Celery Beat	Dispatches the retention cleanup task on a configured interval.
Object storage	MinIO + boto3	S3-compatible private quarantine storage used by the Compose deployment.
File inspection	PyMuPDF + Pillow	Validates PDF/image structure, page count and password protection after signature detection.
Malware gate	ClamAV	Scans the quarantined file before any parser, OCR or verification logic executes.
Identity	JWT + role permissions	Validates issuer/audience/signature and enforces tenant, role and permission boundaries.
Packaging	Docker Compose	Starts MySQL, Redis, MinIO, API, worker and scheduler on one private network.

Primary runtime configuration

Area	Important settings
Upload	MAX_UPLOAD_SIZE_BYTES, MAX_DOCUMENT_PAGES, MAX_FILES_PER_CASE, ALLOW_TIFF_UPLOADS
Storage	STORAGE_BACKEND, MINIO_ENDPOINT, MINIO_BUCKET_NAME, QUARANTINE_STORAGE_PATH
Queue	REDIS_URL, CELERY_DISPATCH_ENABLED
Retention	DOCUMENT_EXPIRY_ENABLED, DOCUMENT_RETENTION_DAYS, DOCUMENT_EXPIRY_SWEEP_SECONDS
Security	SECRET_KEY, JWT_ISSUER, JWT_AUDIENCE, PASSWORD_PROTECTED_PDF_POLICY

2. From API hit to route

FastAPI uses dependency injection at both router and endpoint level. The request does not reach upload logic until the token and rate controls have passed.



Important: the changing port shown in Uvicorn logs is the browser's temporary source port. The API still listens on the fixed server address and port, such as 127.0.0.1:8003.

Route registration

app/main.py creates FastAPI and mounts app/api/v1/router.py at /api/v1. The router then mounts /auth, /cases, /documents, /verification and /reviews. Protected routers share the general API rate-limit dependency; upload routes add the stricter upload dependency.

3. Authentication and rate limiting

Authentication sequence

Step	Control	Failure response
1	HTTPBearer reads Authorization: Bearer <token>.	401 when credentials are absent or invalid.
2	JWT validation checks signature, algorithm, issuer, audience and expiry.	401 Invalid authentication token.
3	Claims become AuthContext: tenant_id, user_id and role.	401 when required claims are invalid.
4	require_permission checks ROLE_PERMISSIONS.	403 Permission denied.
5	Queries include tenant_id so one tenant cannot read another tenant's records.	404 is returned instead of disclosing existence.

How rate limiting is added

rate_limit.py builds fixed-window Redis keys. A Lua script atomically increments the counter, assigns expiry on the first request and returns the current value plus TTL. Atomic execution prevents two simultaneous requests from bypassing the count.

Redis key pattern:
`grd:rate-limit:{scope}:{identity}:{window_id}`

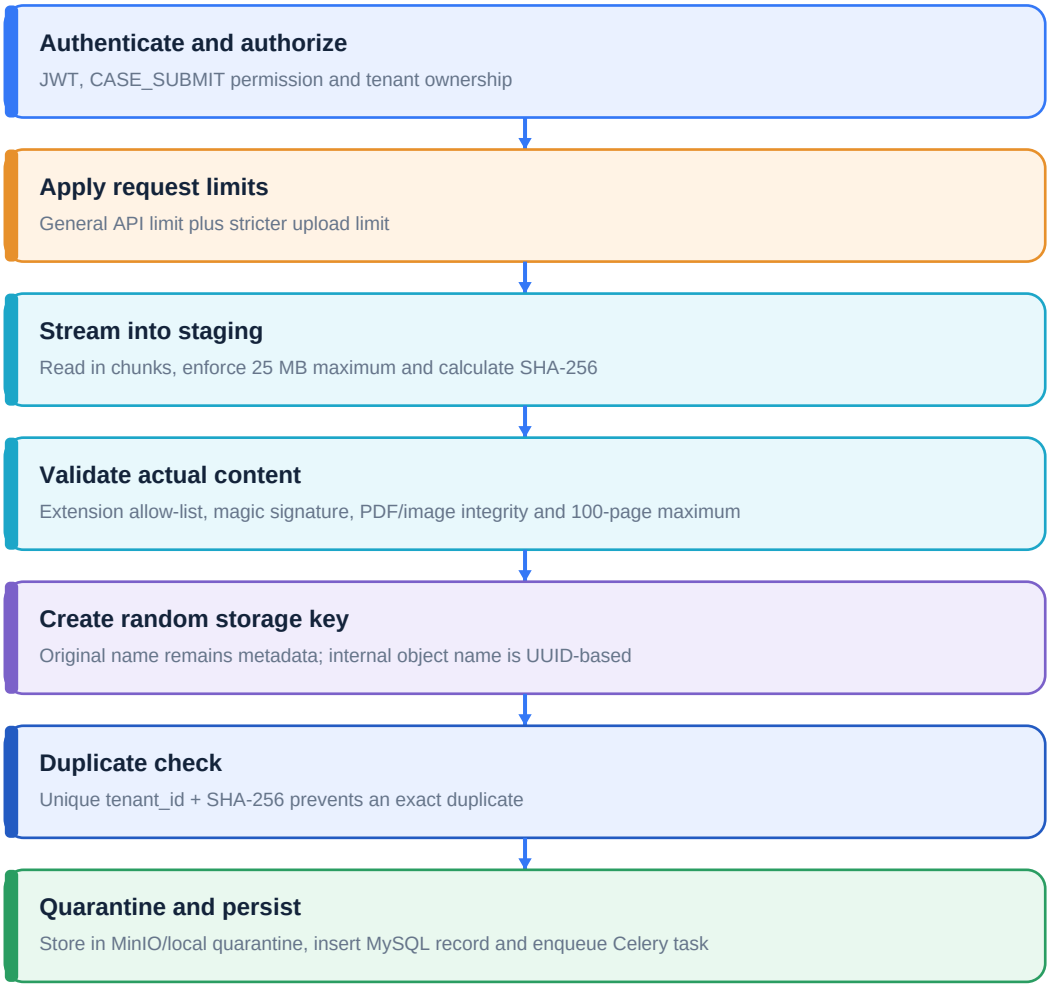
Scopes are api, upload and login. API/upload identities include both tenant+user and tenant-only counters. Login uses the client IP address.

Limit	Default	Window	Purpose
General API per user	120	60 seconds	Prevents one user from flooding protected endpoints.
General API per tenant	1000	60 seconds	Caps aggregate tenant traffic.
Uploads per user	10	600 seconds	Controls expensive file intake.
Uploads per tenant	100	600 seconds	Protects storage and processing capacity.
Login per IP	10	300 seconds	Reduces password guessing.

When exceeded, the API returns 429 with Retry-After and X-RateLimit headers. With fail-open disabled, Redis failure returns 503 instead of silently bypassing the control.

4. Secure document upload pipeline

The browser-provided MIME type is ignored. The application first checks the filename extension and then verifies the file's byte signature and encoded structure.



Accepted type	Signature	Deeper validation
PDF	%PDF-	PyMuPDF opens the file, checks page count, corruption and password policy.
JPEG	FF D8 FF	Pillow decodes all frames and confirms JPEG encoding.
PNG	89 50 4E 47 ...	Pillow decodes image data and confirms PNG encoding.
TIFF	II* or MM*	Only allowed when enabled; Pillow checks frames and page count.

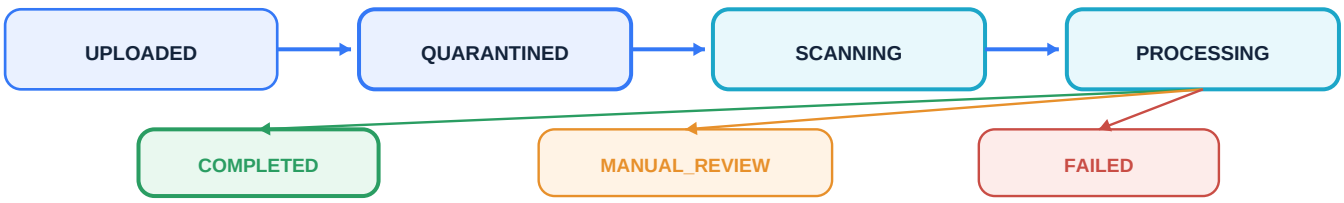
Successful intake returns 202 Accepted. Validation errors return 400/413/415/422; duplicate content returns 409; unavailable storage or queue returns 503.

5. Redis jobs, Celery workers and processing status

Redis does not receive the file itself. Redis carries a small task message containing document_id, case_id, storage_key and password_protected. The file remains in quarantine storage.

Queue sequence

Actor	Action
FastAPI	Calls verify_document.delay(...) after the database record is committed.
Celery client	Serializes the task arguments and publishes the message to Redis.
Redis	Holds the queued message until a worker consumes it.
Celery worker	Receives the task and writes SCANNING to MySQL.
Storage adapter	Returns the local path or downloads a temporary MinIO copy.
ClamAV	Scans before any parser, OCR or extraction step.
Worker	Writes PROCESSING, performs the pipeline, then writes a final state.
Frontend	Polls GET /api/v1/cases every 3 seconds and shows the latest MySQL state.



Retry behavior

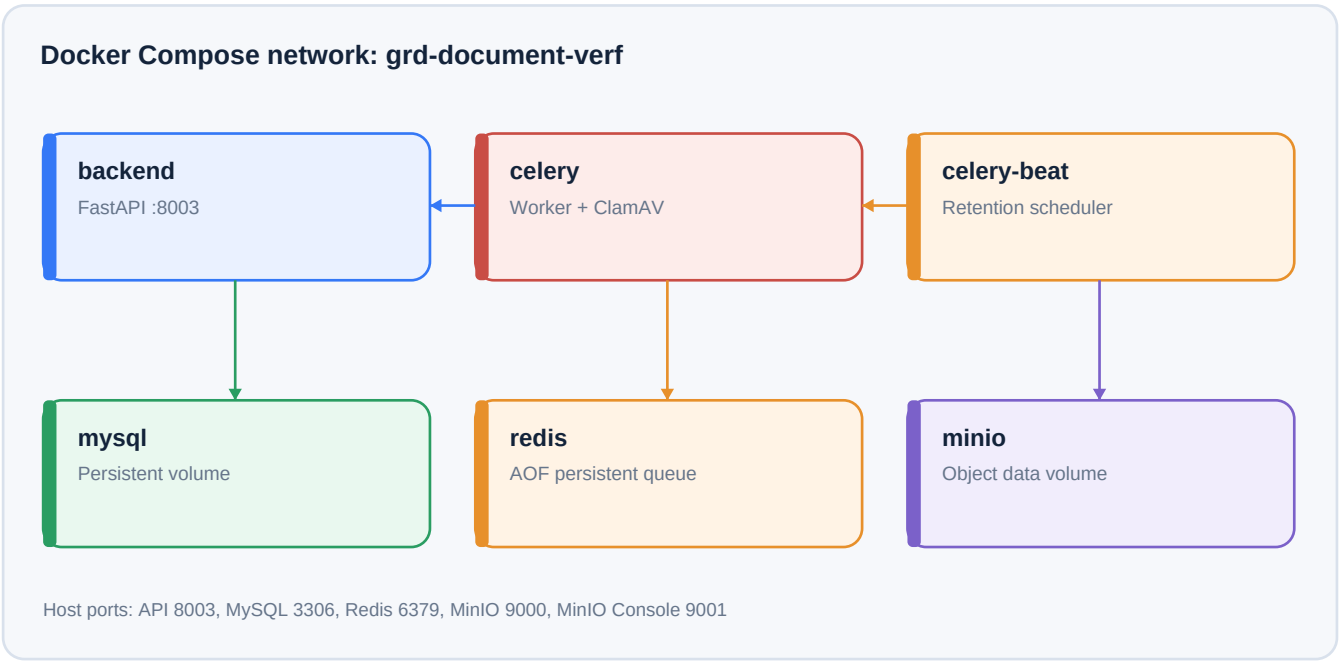
POST /api/v1/documents/{document_id}/retry is tenant-scoped and allowed only for FAILED, non-infected documents whose quarantine object still exists. It locks the database row, resets the document to QUARANTINED/QUEUED, enqueues a new task and restores the previous state if Redis dispatch fails.

Why asynchronous processing is used

ClamAV, OCR and document analysis can be slow. Moving them to Celery keeps API response time short, supports retries, isolates failures and lets worker capacity scale independently from web traffic.

6. How the containers start

docker compose up -d --build creates the private Compose network and persistent volumes, starts infrastructure, waits for health checks, then starts application services in dependency order.



Startup order	What happens
1. MySQL	Loads the grd-mysql-data external volume and becomes healthy after mysqladmin ping succeeds.
2. Redis	Starts with append-only persistence and becomes healthy after redis-cli ping.
3. MinIO	Mounts grd-minio-data, exposes the S3 API/console and passes the live health endpoint.
4. Backend	Runs Uvicorn. Startup creates missing tables and applies the additive document status/storage schema update.
5. Celery	Updates ClamAV definitions, starts worker processes and connects to Redis.
6. Celery Beat	Loads the hourly retention schedule and publishes cleanup tasks to Redis.

Useful commands

```
docker compose up -d --build
docker compose ps
docker compose logs -f backend celery celery-beat minio
docker compose down
```

Do not use docker compose down -v unless the managed volumes are intentionally being removed. MySQL and MinIO data require separate production backups.

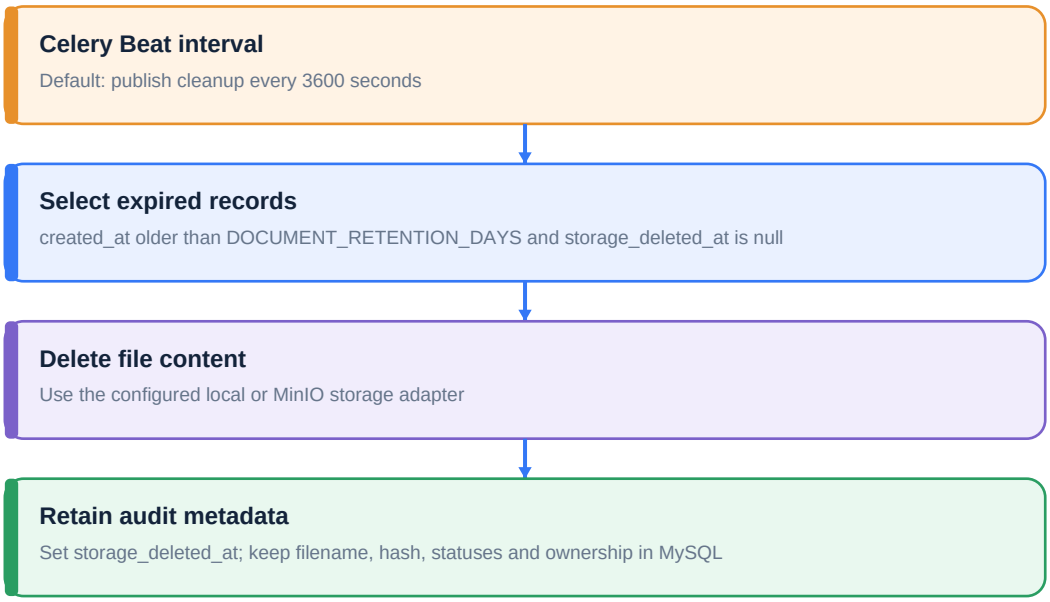
7. Storage architecture and document expiry

Storage adapter

storage.py exposes one interface for local and MinIO operation. Upload validation always uses a temporary local staging file because PyMuPDF, Pillow and ClamAV operate on file paths. After validation, local mode moves the file into the protected quarantine directory; MinIO mode uploads the file and deletes staging.

Operation	Local backend	MinIO backend
Save	Atomic move into storage/quarantine	boto3 upload_file into grd-quarantine
Existence check	Path.is_file()	S3 HeadObject
Worker access	Use protected local path	Download to UUID temporary file
Cleanup	Path.unlink()	S3 DeleteObject
Object name	Random UUID + canonical extension	Same random key; original filename is metadata only

Retention workflow



The dashboard displays Storage: Expired and suppresses retry when the quarantine object has been deleted. A stuck non-final document that reaches retention is safely moved to FAILED.

8. Code map and operational checklist

File	Responsibility
backend/app/main.py	FastAPI creation, startup database initialization, CORS and API mounting.
backend/app/api/v1/router.py	Connects URL prefixes to endpoint modules and general API limits.
backend/app/api/v1/endpoints/cases.py	Case CRUD, secure case upload orchestration and status observations.
backend/app/api/v1/endpoints/documents.py	Direct upload, read/status, retry and deletion endpoints.
backend/app/services/rate_limit.py	Redis Lua counters, limit decisions and response headers.
backend/app/services/storage.py	Signature inspection, SHA-256, local/MinIO storage and temporary materialization.
backend/app/services/antivirus.py	ClamAV command execution and fail-closed scan outcome.
backend/app/workers/tasks.py	Task dispatch, processing stages, malware gate and expiry task.
backend/app/workers/celery_app.py	Redis broker/result configuration and Beat schedule.
backend/app/db/session.py	Engine/session creation, create_all and existing MySQL compatibility update.
docker-compose.yml	All containers, health checks, networks, ports, volumes and environment overrides.
frontend/src/App.tsx	Upload form, case polling, observations, retry and storage availability display.

Pre-production checklist

Check	Required result
Secrets	Replace JWT and MinIO development credentials; use a secret manager.
Transport	Use HTTPS for API and MinIO endpoints.
Object storage	Use redundant/managed MinIO and verified backups; migrate any legacy local objects.
Database	Back up MySQL and confirm the status/storage schema update completed.
Malware	Confirm fresh ClamAV signatures and verify EICAR test rejection.
Queue	Monitor Redis memory/persistence, worker failures and task backlog.
Retention	Confirm the legal retention period and monitor cleanup errors.
End-to-end	Upload a test file and observe QUARANTINED, SCANNING, PROCESSING and final status.

System rule: MySQL stores the authoritative business state, MinIO stores file bytes, Redis coordinates short-lived counters and tasks, and Celery performs background work. The file itself is never sent through Redis.